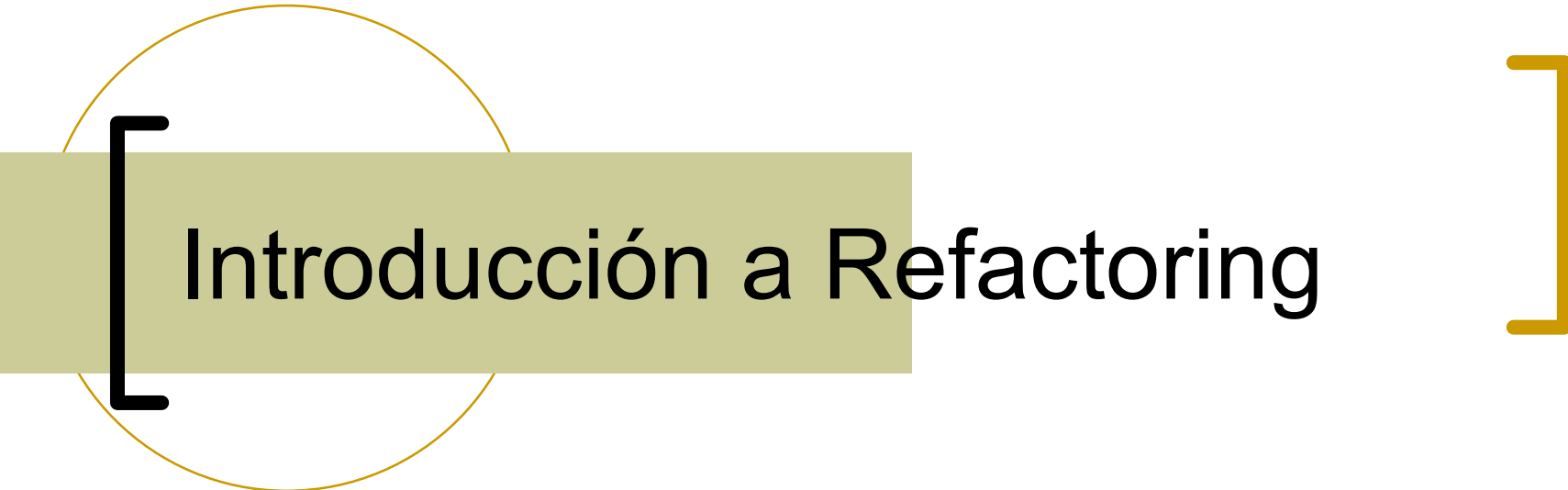




Introducción a Refactoring

Dra. Alejandra Garrido

Objetos 2 – Fac. De Informática – U.N.L.P.

alejandra.garrido@lifa.info.unlp.edu.ar

[Cambios



[Leyes de Lehman]

- Continuing Change (1974)
 - Los sistemas deben adaptarse continuamente o se vuelven progresivamente menos satisfactorios
- Continuing Growth (1991)
 - la funcionalidad de un sistema debe ser incrementada continuamente para mantener la satisfacción del cliente
- Increasing Complexity (1974)
 - A medida que un sistema evoluciona su complejidad se incrementa a menos que se trabaje para evitarlo
- Declining Quality (1996)
 - La calidad de un sistema va a ir declinando a menos que se haga un mantenimiento riguroso

[Costo del mantenimiento]

- Mantenimiento
 - correctivo, evolutivo, adaptativo, perfectivo, preventivo.
- **Entender código existente:**
50% del tiempo de mantenimiento



Lidiando con el código spaguetti

```
if (evt1.AbsoluteTime < evt2.AbsoluteTime) {
    return -1;
} else if (evt1.AbsoluteTime > evt2.AbsoluteTime) {
    return 1;
} else {
    // a igual valor de AbsoluteTime, los channelEvent tienen prioridad
    if (evt1.MidiEvent is ChannelEvent && evt2.MidiEvent is MetaEvent) {
        return -1;
    } else if (evt1.MidiEvent is MetaEvent && evt2.MidiEvent is ChannelEvent) {
        return 1;
    }
    // si ambos son channelEvent, dar prioridad a NoteOn == 0 sobre NoteOn > 0
    if (evt1.MidiEvent is ChannelEvent && evt2.MidiEvent is ChannelEvent) {

        chanEvt1 = (ChannelEvent) evt1.MidiEvent;
        chanEvt2 = (ChannelEvent) evt2.MidiEvent;

        // si ambos son NoteOn
        if (chanEvt1.EventType == ChannelEventType.NoteOn
            && chanEvt2.EventType == ChannelEventType.NoteOn) {

            // chanEvt1 en NoteOn(0) y el 2 es NoteOn(>0)
            if (chanEvt1.Arg1 == 0 && chanEvt2.Arg1 > 0) {
                return -1;
            }
            // chanEvt1 en NoteOn(0) y el 2 es NoteOn(>0)
            } else if (chanEvt2.Arg1 == 0 && chanEvt1.Arg1 > 0) {
                return 1;
            }
        } else {
            return 0;
        }
    }
}
```

[Lidiando con el código de la IA Gen]

```
public String prettyPrint(List<String> atributos) {  
    StringBuilder sb = new StringBuilder();  
    for (String atributo : atributos) {  
        switch (atributo) {  
            case "nombre":  
                sb.append(nombre).append(" - ");  
                break;  
            case "extension":  
                sb.append(extension).append(" - ");  
                break;  
            case "tamano":  
                sb.append(tamano).append(" - ");  
                break;  
            case "fechaCreacion":  
                sb.append(fechaCreacion).append(" - ");  
                break;  
            case "fechaModificacion":  
                sb.append(fechaModificacion).append(" - ");  
                break;  
            case "permisos":  
                sb.append(permisos).append(" - ");  
                break;  
        }  
    }  
}
```

[Big Balls of Mud]

Brian Foote & Joe Yoder. 2000. <http://www.laputan.org/mud/>



Like a set of dominoes, the former home of the Atlanta Braves collapsed Saturday (Courtesy WXIA)

[Diseñar es difícil!

- Los elementos distintivos de la arquitectura de un sistema no surgen hasta *después* de tener código que funciona
- No se trata sólo de agregar, sino de *adaptar, transformar, mejorar*
- Construir el sistema perfecto es imposible
- Los errores y el cambio son inevitables
- Hay que aprender del **feedback**

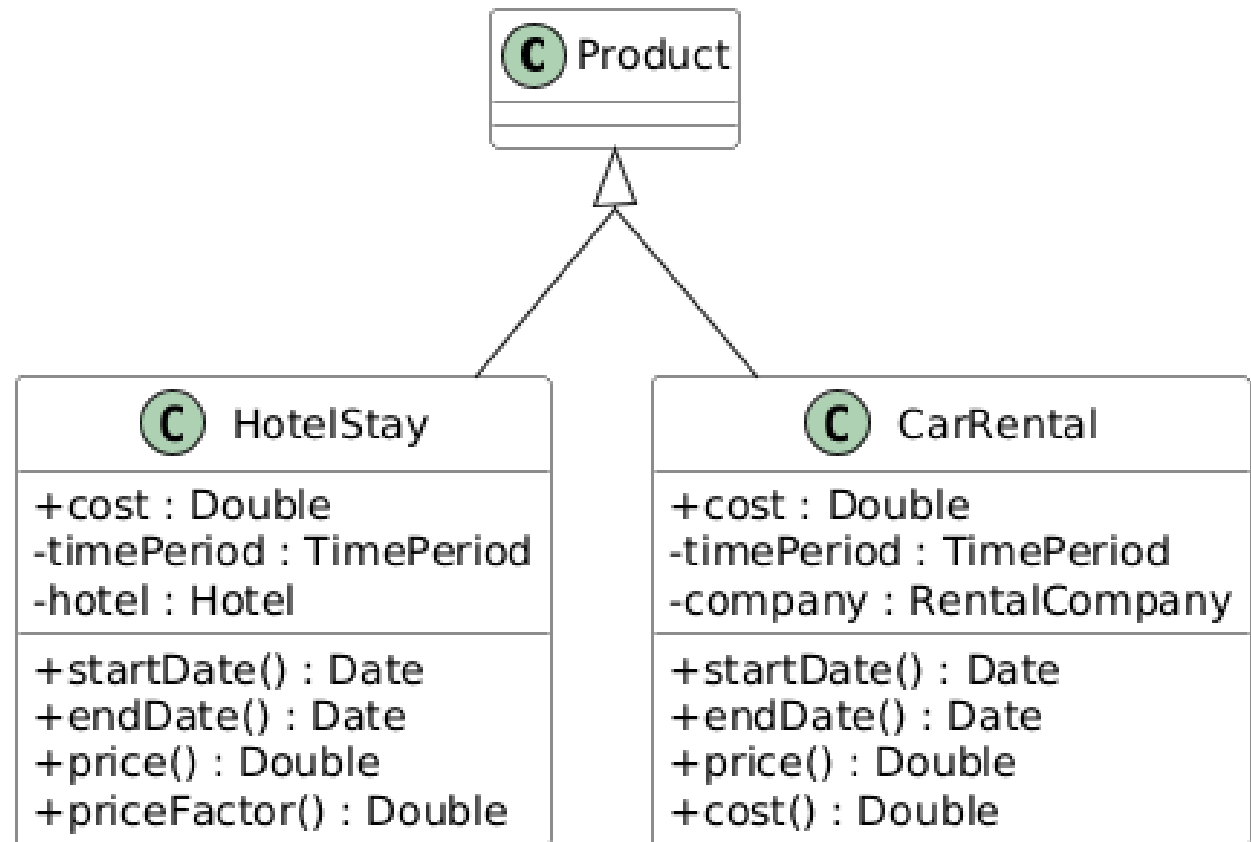


[La iteración es fundamental]

- “Reusable software is the result of many design iterations. Some of these iterations occur after the software has been reused”

(Bill Opdyke. 1992)

[Ejemplo



¿Product es abstracta o concreta?

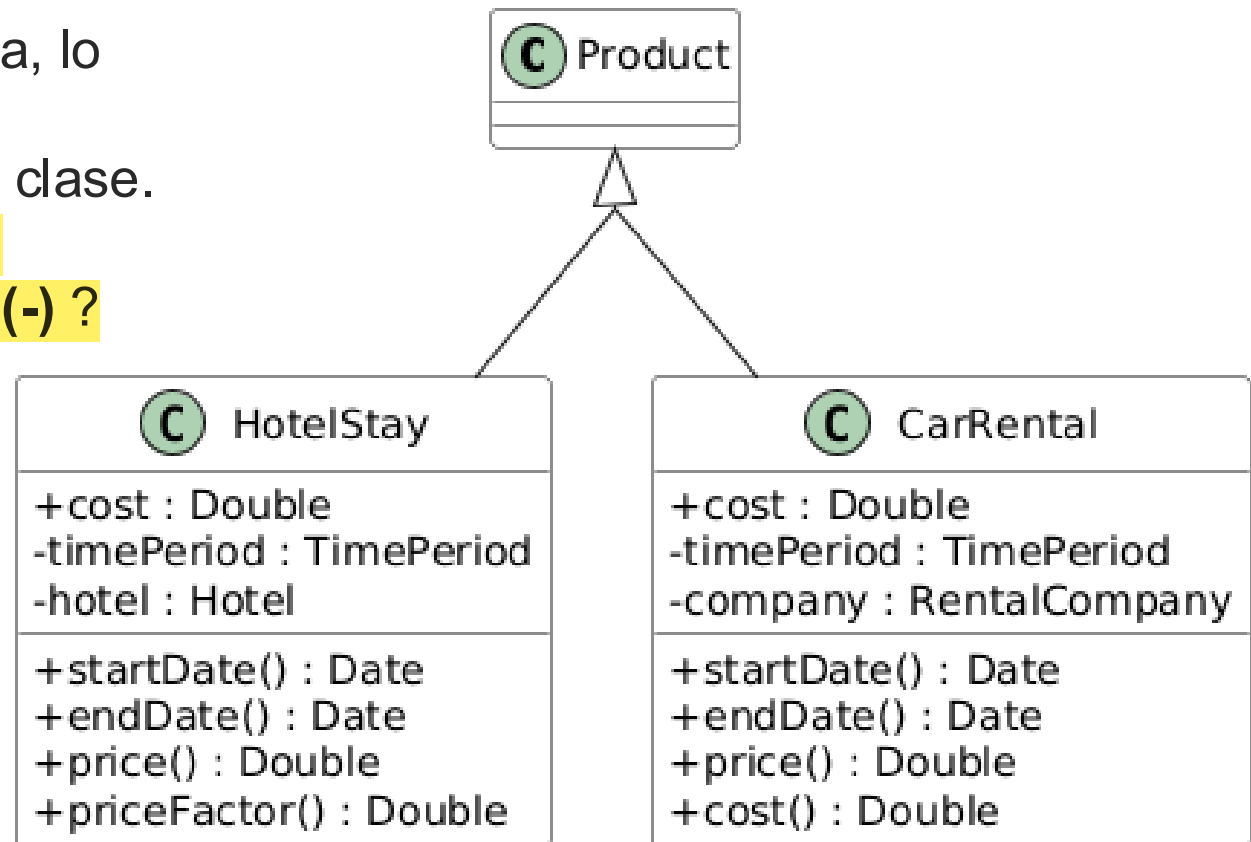
¿HotelStay y CarRental son polimórficas?

¿Cómo podría implementar en Java la variable **cost**?

¿Hay alguna heurística o mal olor con relación a esto?

La variable **cost** está declarada como pública, lo que rompe el encapsulamiento de la clase.

¿Se puede cambiar la declaración a **privada (-)** ?

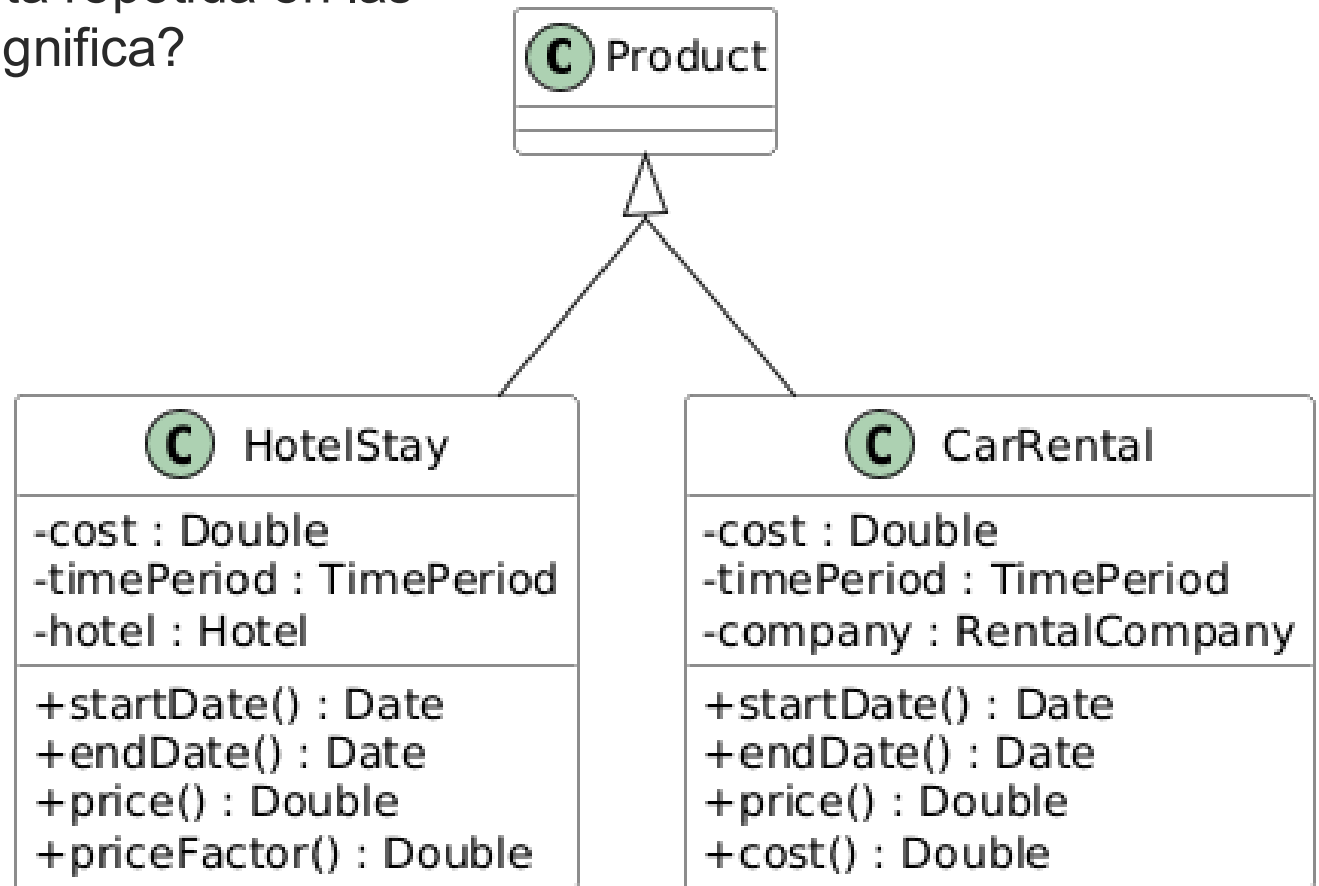


∄ refs a la variable ⇒ el código es válido (compila y pasa los test de unidad)

∃ refs a la variable ⇒ el código no es válido (no compila). ¿Qué hacemos?

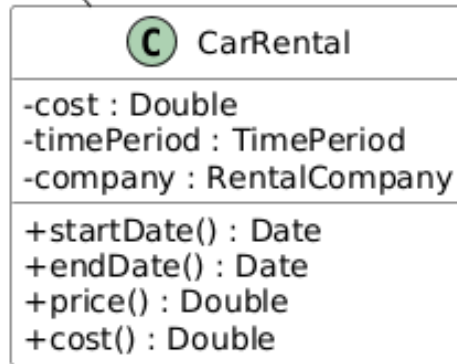
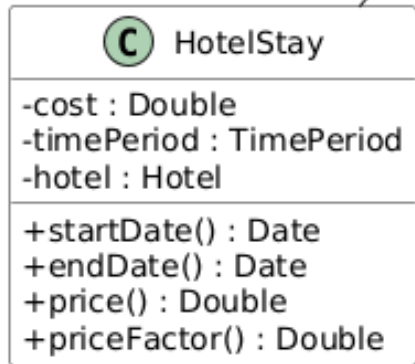
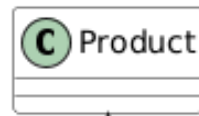
✖ implementar accessors + cambiar referencias por invocaciones a métodos

La variable **cost** está repetida en las subclases. ¿Qué significa?



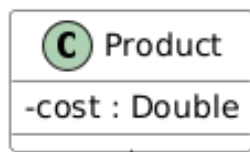
Si todo lo que sea un Product va a tener costo

⇒ se puede “generalizar” la variable (moverla hacia Product)



¿Qué implica mover la variable cost?

HotelStay	startDate() { return timePeriod.start; }	endDate() { return timePeriod.end; }
	price() { return timePeriod.duration() * hotel.nightPrice() * hotel.discountRate();} priceFactor() { return this.cost() / this.price(); }	
CarRental	startDate() { return timePeriod.start; }	endDate() { return timePeriod.end; }
	price() { return company.price() * company.promotionRate(); } cost() { return this.cost(); }	



“Empujamos para arriba” la declaración de **cost**.

¿El resultado es el mismo que antes?

Debemos declarar a **cost** como protegida (#)

HotelStay	startDate() { return timePeriod.start; }	endDate() { return timePeriod.end; }
	price() { return timePeriod.duration() * hotel.nightPrice() * hotel.discountRate();} priceFactor() { return this.cost() / this.price(); }	
CarRental	startDate() { return timePeriod.start; }	endDate() { return timePeriod.end; }
	price() { return company.price() * company.promotionRate(); } cost() { return this.cost(); }	

[Refactoring. Definición]

Refactoring (sustantivo): es una **transformación** que se realiza en la estructura interna del software para hacerlo fácil de entender y más barato de modificar, y que **preserva el comportamiento** observable

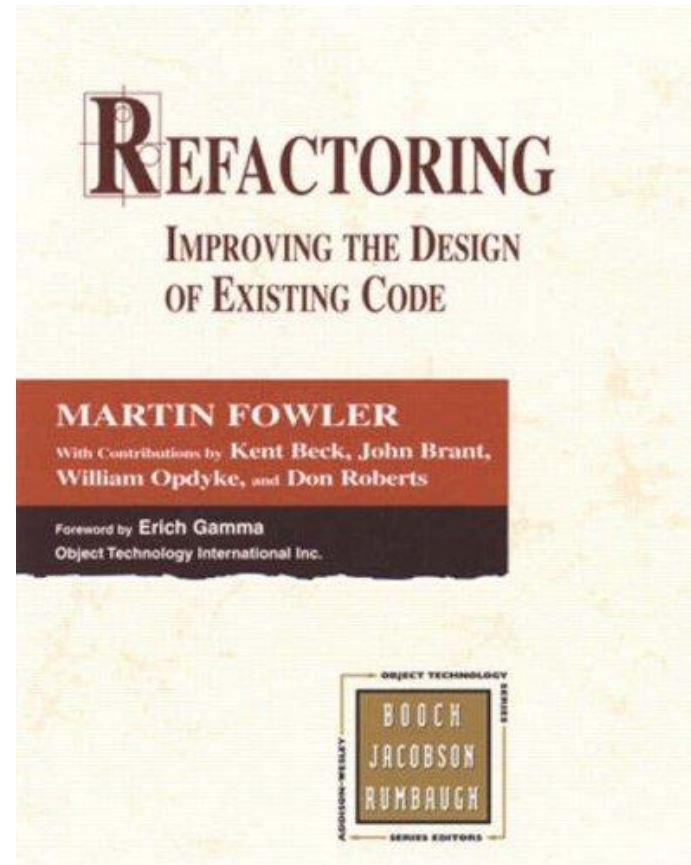
- Con un nombre específico con el que está catalogada y una **secuencia de pasos** ordenados (“mechanics”)



[Refactoring como un proceso]

Es el proceso a través del cual se cambia un sistema de software

- para **mejorar** la organización, legibilidad, adaptabilidad y mantenibilidad del código luego que ha sido escrito
- que **NO altera** el comportamiento externo del sistema



[Precondiciones y mecanismo]

- Si el comportamiento cambia (no compila, no pasa los tests), entonces el cambio NO fue un refactoring
- Debemos tener cuidado de no introducir un bug
- Los refactorings tienen:
 - precondiciones
 - que chequean que se pueda aplicar
 - mecanismo de aplicación
 - pasos que *cuidan* la preservación del comportamiento

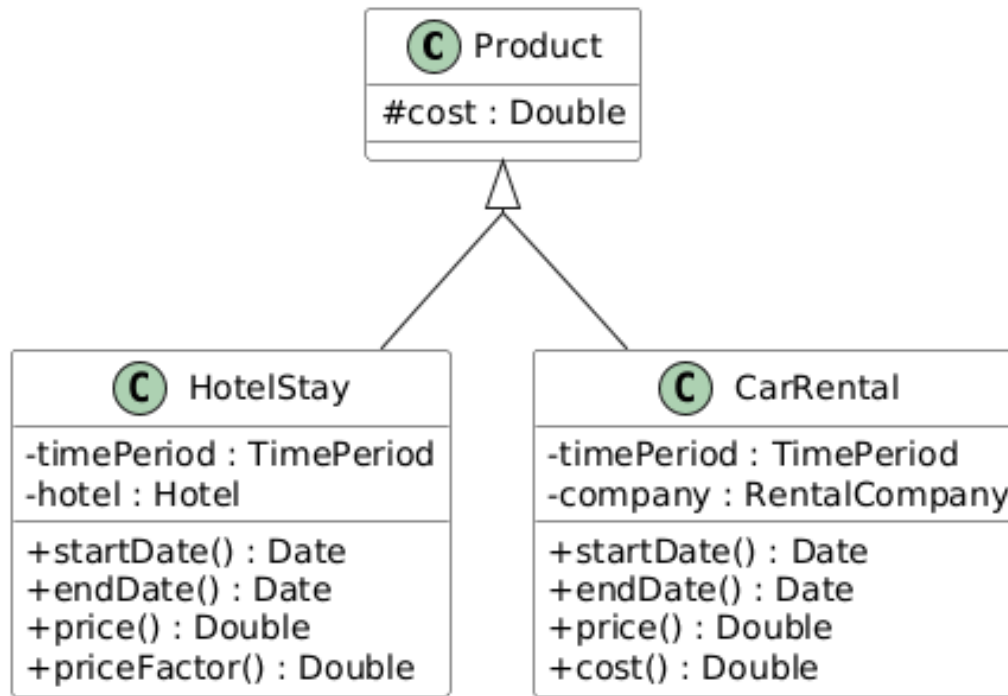
[Refactoring: Pull Up Field]

■ Precondiciones

- La v.i. se usa en las subclases de la misma manera (semánticamente representan lo mismo)
- La v.i. comparte el mismo nombre en las subclases (si no se la renombra antes) y el mismo tipo
- La v.i. con ese nombre no debe existir en la superclase.

■ Mecánica

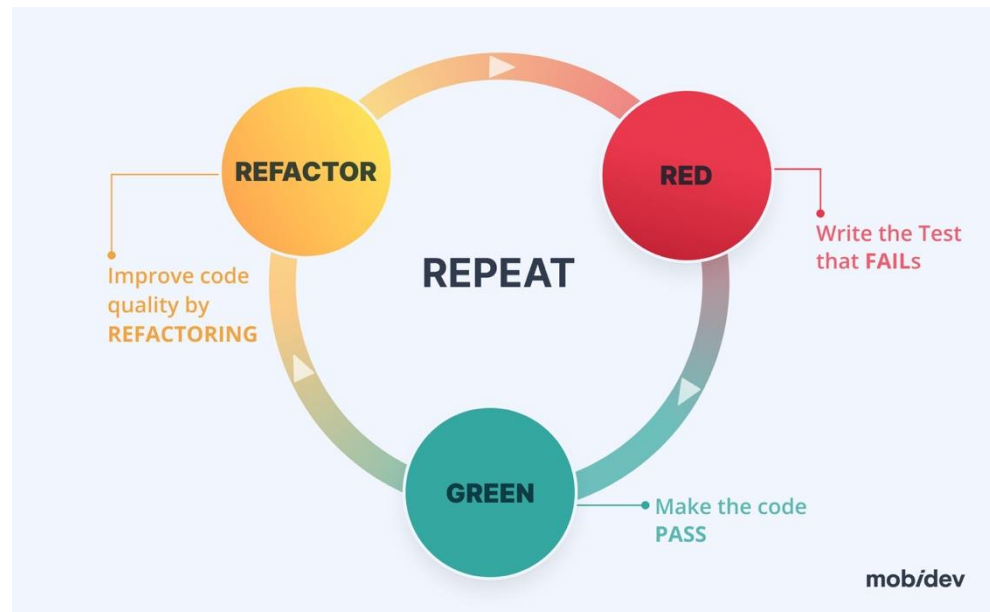
- Crear una nueva v.i. en la superclase
 - Si la v.i. en las subclases es privada, declarar a la v.i. como *protected* para que las subclases puedan referenciarla
- Borrar la v.i. de las subclases
- Compilar y testear



HotelStay	startDate() { return timePeriod.start; }	endDate() { return timePeriod.end; }
	price() { return timePeriod.duration() * hotel.nightPrice() * hotel.discountRate();}	
	priceFactor() { return this.cost() / this.price(); }	
CarRental	startDate() { return timePeriod.start; }	endDate() { return timePeriod.end; }
	price() { return company.price() * company.promotionRate(); }	
	cost() { return this.cost(); }	

[Aplicamos un refactoring]

- Hemos transformado las definiciones de las clases (cambio en el código fuente)
- Hemos preservado la funcionalidad
- Deberíamos tener tests de unidad que aseguren esto





El gerente se ha dado cuenta, mirando el método *priceFactor()*, que la variable **cost** se debería renombrar a **quote**.

```
3 public float priceFactor(){  
4     return this.cost / this.price;  
5 }
```

Ejercicio para la próxima clase:

- ¿Cuáles son las precondiciones?
- ¿Cuál es la mecánica? Es decir, qué deberíamos cambiar y en qué orden para que el programa siga compilando y funcionando igual.

[Características del Refactoring]

■ Implica

- Eliminar duplicaciones
- Simplificar lógicas complejas
- Clarificar códigos

■ Cuándo

- Una vez que tengo código que funciona y **pasa los tests**
- A medida que voy desarrollando:
 - cuando encuentro código difícil de entender (ugly code)
 - cuando tengo que hacer un cambio y necesito reorganizar primero
- Antes de llegar a



■ Testear después de cada cambio

[Cómo ayuda el refactoring?]

- Introduce mecanismos que solucionan problemas de diseño
- A través de cambios **pequeños**
 - Hacer muchos cambios pequeños es más fácil y más seguro que un gran cambio
 - Cada pequeño cambio pone en evidencia otros cambios necesarios
- ¿Cómo encontrar los lugares donde conviene aplicar refactoring (para que el código sea más legible y mantenible)?

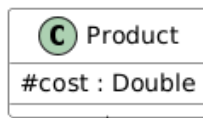
[BAD SMELLS!]

- Indicios de problemas que requieren la aplicación de refactorings
- Posteriormente llamados **CODE SMELLS**



[Code smells de Objetos 1?]

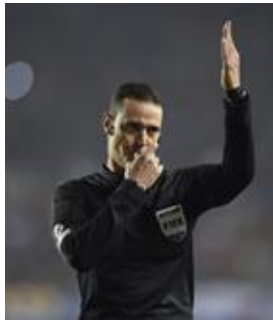
- Romper encapsulamiento
- Código duplicado
- Clase larga
- Método largo
- Switch statements
- Clase de datos
- No “es un”
- ...



Algún code smell?

Código duplicado! ⇒ Pull up Method

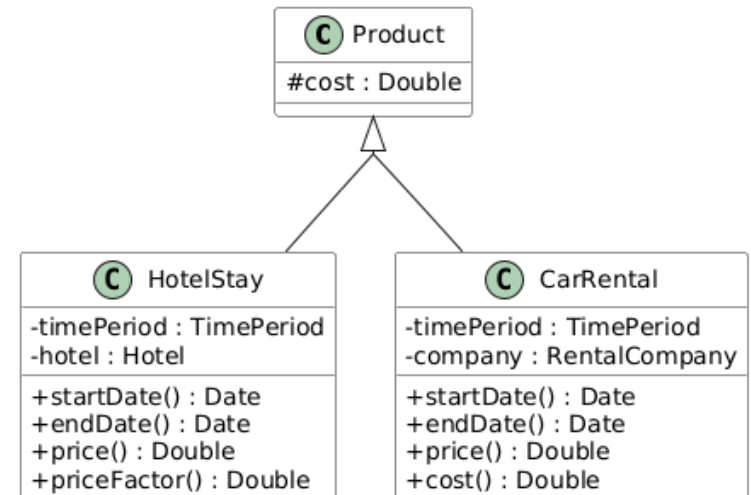
HotelStay	startDate() { return timePeriod.start; }	endDate() { return timePeriod.end; }
	price() { return timePeriod.duration() * hotel.nightPrice() * hotel.discountRate();} priceFactor() { return this.cost() / this.price(); }	
CarRental	startDate() { return timePeriod.start; }	endDate() { return timePeriod.end; }
	price() { return company.price() * company.promotionRate(); } cost() { return this.cost(); }	



[Pull Up Method]

■ Precondiciones

- El cuerpo de los métodos deben ser idéntico
- La signatura debe ser idéntica (si no se renombra antes)
- Superclase común
- Que no haya conflicto con métodos de la superclase
- Que todos los elementos a los que accede el método sean accesibles desde la superclase



[Pull Up Method (p. 322 @ Refactoring, Fowler)]

🔗 Consideraciones de **startDate()**

- a. Las implementaciones son iguales
- b. Existe una superclase: *Product*
- c. *Product* no tiene un método con ese nombre
- d. $\exists$ referencias a la variable *timePeriod* $\Rightarrow$ realizar PullUp Field de *timePeriod* previamente

🔗 Mecánica (adaptada a este ejemplo)

- a. Cree un método en la superclase (*Product*) y copie el cuerpo del método: *startDate()*
- b. Borre el método en una de las subclases
- c. Compile y corra los test
- d. Siga borrando el método en cada subclase, compilando y testeando hasta que solo quede el método en la superclase *Product*.
- e. Verifique que en las llamadas al método el tipo (de parámetros y variables) se ajuste al tipo de la superclase.

[Resumen]

✧ Pasos de un refactoring

- ✧ Verificar precondiciones
- ✧ Ejecutar transformaciones
- ✧ Compilar&Testear

✧ Refactoring presentados

- ✧ Encapsulate Field (p.206)
- ✧ Pull Up Field (p.320)
- ✧ Pull Up Method (p.322)